



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4306 • Design & Analysis of Algorithms • Fall 2026

Counting Comparisons, Measuring Time

Instrumenting code, timing it honestly, and the doubling question

Session 2 of 24 • Tuesday 22 September 2026 • 2:30 – 3:50 PM

• Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 2 OF 24

Counting Comparisons, Measuring Time

Tuesday, 22 September 2026 • 2:30 – 3:50 PM

LEARNING OBJECTIVES

- ✓ Instrument a function so that it reports how much work it did.
- ✓ Explain why we count comparisons rather than seconds when comparing algorithms.
- ✓ Time a function properly in Python, and read a timing table as a set of *reactions*.
- ✓ Apply the doubling question — what happens to the cost when n doubles? — to unfamiliar code.

FROM SESSION 1

Bakary removes one name per step; Chantal removes half the book. We asserted that this produces a difference of a hundred million at a billion names. Today we stop asserting and start measuring.

1 • Make the Code Confess 25 min

The cleanest way to find out how much work a function does is to ask it. We add a counter that is incremented at exactly the place where the real work happens — here, the comparison against the target — and return it alongside the answer.

PY LINEAR_COUNT

```
def linear_count(A, x):  
    """Same as linear_search, but reports how many comparisons it made."""  
    comparisons = 0  
    for i in range(len(A)):  
        comparisons += 1  
        if A[i] == x:  
            return i, comparisons  
    return -1, comparisons
```

PY BINARY_COUNT

```
def binary_count(A, x):  
    comparisons = 0  
    lo, hi = 0, len(A) - 1  
    while lo <= hi:  
        mid = (lo + hi) // 2  
        comparisons += 1  
        if A[mid] == x:  
            return mid, comparisons  
        if A[mid] < x:  
            lo = mid + 1  
        else:  
            hi = mid - 1  
    return -1, comparisons
```

Where to put the counter

Not at the top of the function, and not at the bottom — those would count calls, not work. The counter goes where the **repeated** operation happens, inside the loop. Choosing that operation is a modelling decision, and we are choosing comparisons because they are what both functions actually spend their time on.

1.1 Running them on the worst case

An input's size does not determine its cost by itself. Searching for the first element is easy for Bakary and hard for nobody. To compare fairly we must fix which input we are talking about, and the honest choice is the one that costs the most: the target sitting at the very end, or absent entirely.

PY THE WORST CASE FOR BOTH

```
for n in [10, 1_000, 1_000_000]:  
    A = list(range(n))          # sorted: 0, 1, 2, ... n-1
```

```
target = n - 1 # the worst possible place to hide it
print(n, linear_count(A, target)[1], binary_count(A, target)[1])
```

COMPARISON COUNTS

n	linear	binary
10	10	4
1,000	1,000	10
1,000,000	1,000,000	20

WHY COUNTING BEATS TIMING — FOR NOW

A comparison count is a property of the **algorithm**. It is identical on your laptop, on mine, and on a machine built in 1975. A number of seconds is a property of the machine, the language, the operating system's mood, and whether someone else was compiling at the time. We will time things in a moment, but counting is what we reason with.

1.2 The reaction, not the number

Look at the columns as *reactions* to a change in n rather than as values.

n multiplied by	linear count multiplied by	binary count changed by
10 $\rightarrow$ 1,000 ($\times 100$)	$\times 100$	+6 questions
1,000 $\rightarrow$ 10^6 ($\times 1,000$)	$\times 1,000$	+10 questions
any doubling of n	$\times 2$	+1 question

That last row is the one to memorise. Linear search **multiplies** when the input doubles; binary search **adds one**. Multiplication and addition are different kinds of animal, and the gap between them widens for ever.

2 · Putting a Clock on It 25 min

Counting is a model. Models can be wrong, so we check this one against reality. Python's `time.perf_counter()` is the right tool: it is monotonic and has high resolution, unlike `time.time()`, which tracks the wall clock and can jump.

PY A TIMING HARNESS

```
import time
```

```
def timed(fn, A, x, repeats=1):
    """Average milliseconds per call of fn(A, x)."""
    start = time.perf_counter()
    for _ in range(repeats):
        fn(A, x)
    elapsed = time.perf_counter() - start
    return elapsed * 1000 / repeats
```

Why `repeats` exists

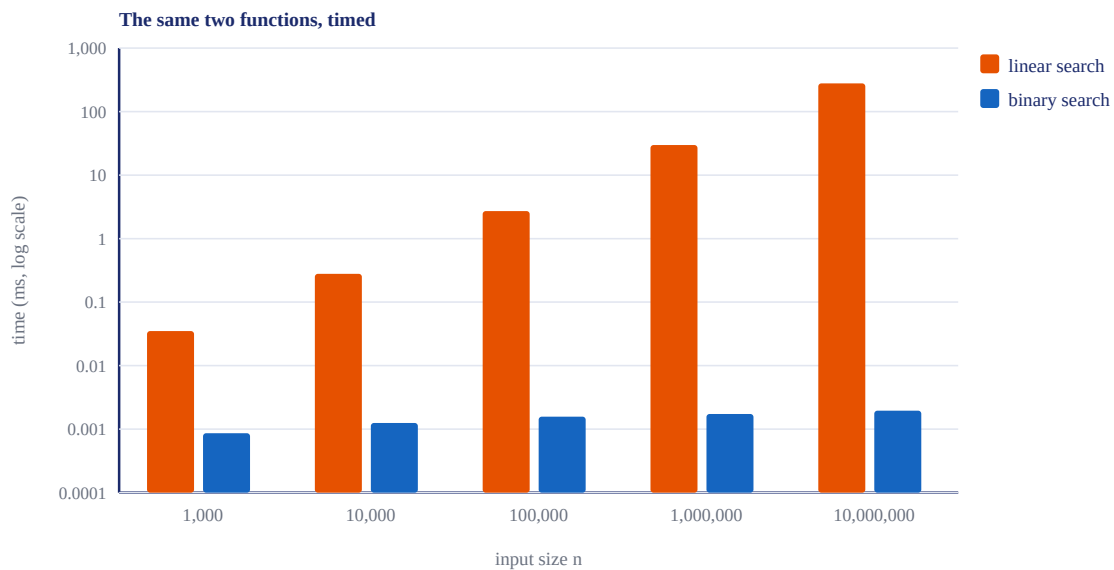
One call to `binary_search` takes about two microseconds. The clock itself has a resolution of roughly the same order, so measuring a single call measures mostly noise. Running it 200,000 times and dividing gives a number you can trust. Any timing of a fast function that does not repeat it is not a measurement; it is a rumour.

PY TIMING BOTH AT FIVE SIZES

```
for n in [1_000, 10_000, 100_000, 1_000_000, 10_000_000]:
    A = list(range(n))
    x = n - 1
    print(n,
          timed(linear_search, A, x),
          timed(binary_search, A, x, repeats=200_000))
```

MEASURED ON THE LECTURE MACHINE

n	linear (ms)	binary (ms)
1,000	0.035	0.000863
10,000	0.279	0.001250
100,000	2.708	0.001574
1,000,000	29.733	0.001736
10,000,000	277.280	0.001953



Two different animals. The vertical scale is logarithmic, so equal steps up the axis mean equal multiplications. The orange bars climb in a straight staircase — each tenfold increase in n costs tenfold more time. The blue bars barely move: a ten-thousand-fold increase in input takes binary search from 0.00086 ms to 0.00195 ms, a factor of 2.3.

2.1 Reading the table honestly

What matches the model

Linear search multiplies by roughly ten at each step, exactly as the comparison count predicted. Binary search grows by about one step per doubling: from 10^3 to 10^7 is a little over thirteen doublings, and the time rose by a factor of 2.3 — not thirteen-fold, because the fixed cost of calling the function dominates at this scale.

What does not

Every number in this table will change on your machine, and that is expected, not a bug. Some of the linear numbers even grow slightly faster than tenfold — memory caching, not algorithmics. Measurement contains truth and noise together, and part of the skill is knowing which is which.

THE GAP BETWEEN MEASUREMENT AND KNOWLEDGE

A measurement tells you about a machine on a Tuesday. We want statements about an **algorithm**, true everywhere and for ever. That is exactly the gap Session 3 closes — and now you know what it is a gap between, which is why we spent today filling it with real numbers.

3 · The Doubling Question 15 min

Here is a diagnostic tool you can apply immediately, even to code you did not write and cannot read. Run it on an input of size n , then on an input of size $2n$, and compare.

Time when n doubles	What the code is probably doing	Example
unchanged	the work does not depend on the input size	reading A[0]
one step more	repeatedly halving something	binary search
twice as long	one pass over the data	linear search, summing a list
four times as long	a loop inside a loop, both over the data	comparing every pair
squared	trying every possible subset or arrangement	brute-force scheduling

You do not need to understand a program to run this test on it. That makes it the most practical diagnostic in the course, and it will be your first move for the rest of the term.

PY APPLY THE TEST TO CODE YOU HAVE NOT ANALYSED

```
def mystery(A):
    total = 0
    for i in range(len(A)):
        for j in range(len(A)):
            total += A[i] * A[j]
    return total

for n in [500, 1000, 2000, 4000]:
    A = list(range(n))
    print(n, timed(mystery, A, None))    # what do you predict before running it?
```

Predict first, then run

Write your prediction down before executing. A prediction that turns out wrong teaches you something; a number you merely observe teaches you nothing. This will be a standing habit in this course.

SESSION 2 TAKEAWAY

Instrument the code to count its own work, because a count belongs to the algorithm while a second belongs to the machine. Then check the count against a clock, repeating fast functions enough times to rise above the noise. Read every table as a set of reactions, and when in doubt ask the one question: **what happens when I double the input?**

IN-CLASS EXERCISES (work in pairs)

1. Predict the output of the `mystery` timing loop above, then run it. By what factor does the time grow at each doubling, and what does that tell you?
2. Modify `linear_count` to search for an element that is *present at a random position* rather than absent. Run it 1,000 times on a list of 1,000 and average the comparison count. What number do you get, and why that one?
3. Why would timing `binary_search` with `repeats=1` produce a misleading result? Try it and report what you see.
4. Someone reports that their sorting program takes 2 seconds on 10,000 items, so it will take 20 seconds on 100,000. What assumption are they making, and how would you test it in two minutes?
5. Write a function whose running time is *unchanged* when the input doubles, and one whose time grows fourfold. Verify both by measurement.

Homework · due Friday 25 September

1. Reproduce the comparison-count table of §1.1 on your machine and confirm every entry.
2. Reproduce the timing table of §2 for all five sizes. Submit your numbers beside mine, and one paragraph on which quantities differ and which *ratios* do not.
3. Write `sum_list(A)`, `max_list(A)` and `count_pairs(A)` (the last one comparing every pair). Apply the doubling test to each at $n = 1000, 2000, 4000, 8000$ and classify each by its reaction.
4. Take the `binary_count` function and plot comparison count against n for $n = 2, 4, 8, \dots, 4096$ with a target that is absent. Describe the shape of the plot in one sentence.
5. Reading: CLRS §2.2. Half a page on why we analyse the worst case rather than the average.

Next session · Friday 25 September: We finally name what we have been measuring. Big-O notation — the three simplification moves, the gallery of growth rates, and why a faster computer cannot rescue a badly shaped algorithm.